复习

1）如何判定一个对象是否是垃圾

引用计数法：会导致循环引用

可达性分析

GCRoot：线程、常量、静态变量、类加载器（尽量选择生命周期长的对象）

当一个对象被标记成垃圾之后，它是非死不可吗？

1）重写finalize方法

2）有且仅有一次改过自新的机会，且还没有执行过finalize方法

2）垃圾回收算法

1、标记清除算法-->老年代

2、复制算法--->新生代

3、标记整理---->老年代

4、分代收集算法

3）垃圾回收器

1. serial垃圾器
    1. 复制算法
    2. 单线程
2. ParNew
    1. 复制算法
    2. 并行收集器
3. ParallelScavenge
    1. 复制算法
    2. 并行收集器
    3. 关注吞吐量
4. Serial Old
    1. 标记整理算法
    2. 单线程
5. Parallel Old
    1. 标记整理算法
    2. 并行收集器
    3. 关注吞吐量
6. CMS
    1. 标记清除算法
    2. 并发收集器
    3. 关注最短停顿时间
    4. 分为4个步骤
        1. 初始标记（STW）
        2. 并发标记

        3. 重新标记（STW）
        4. 并发清除
7. G1
    1. 分代收集算法
    2. 并发收集器
    3. 关注最短停顿时间
    4. 分为4个步骤
        1. 初始标记（STW）
        2. 并发标记
        3. 最终标记（STW）
        4. 筛选回收（STW）
    5. 将堆内存划分为一个个的region区

目标

1）JVM有哪些参数

2）5大常用命令

3）性能调优的步骤

1 JVM参数

1.1 标准参数

就是不会随着jdk版本的变化而变化的参数

java -version

1.2 -X参数

非标准参数

就是会随着JDK版本的变化，有可能会变

-Xint ：解释器模式

-Xcomp：编译器模式（JIT）

-Xmixed

jdk编译器不一样

1.3 -XX参数

1）Boolean类型

格式：-XX:[+/-]

例子：-XX:+UseG1GC -XX:-UseG1GC

-XX:+UseConcMarkSweepGC

2）非Boolean

格式：

-XX:=

例子：-XX:MaxHeapSize=100M

-XX:IntialHeapSize=100M

1.4 其他参数

都可以通过-XX参数去实现

-Xmx100M ：设置最大堆内存 ==-XX:MaxHeapSize=100M

-Xms100M:初始化堆内存 == -XX:IntialHeapSize=100M

-Xss1M：栈的大小 == -XX:ThreadStackSize=1M

1.5 如何查看参数

> java -XX:+PrintFlagsFinal -version

manageable：当参数被标记为这个类型的时候，表示这个参数能够热修改

:= ---->表示该参数不是默认值，已经被用户或者JVM自身修改过的值

1.6 设置参数的方式

刚刚讲的这两种方式都是开发过程中修改

web容器比如tomcat，可以在脚本中的进行设置，比如bin目录下的xxx.sh/catalina.sh----->JVM参数

1.7 常用的参数

别人给到你一个参数，你得知道这个参数是干什么的

然后你也大概知道一些参数可以控制什么东西

参数	含义	说明
-XX:InitialHeapSize=100M	初始化堆大小	简写-Xms100M
-XX:MaxHeapSize=100M	最大堆大小	简写-Xmx100M
-XX:NewSize=20M	设置年轻代的大小	
-XX:MaxNewSize=50M	年轻代最大大小	
-XX:OldSize=50M	设置老年代大小	
-XX:MetaspaceSize=50M	设置方法区大小	
-XX:MaxMetaspaceSize=50M	方法区最大大小	
-XX:+UseParallelGC	使用UseParallelGC	新生代，吞吐量优先
-XX:+UseParallelOldGC	使用UseParallelOldGC	老年代，吞吐量优先
-XX:+UseConcMarkSweepGC	使用CMS	老年代，停顿时间优先
-XX:+UseG1GC	使用G1GC	新生代，老年代，停顿时间优先
-XX:NewRatio	老生代和新生代的比值	比如-XX:Ratio=4，则表示新生代:老年代=1:4，也就是新生代占整个堆内存的1/5
-XX:SurvivorRatio	两个S区和Eden区的比值	比如-XX:SurvivorRatio=8，也就是(S0+S1):Eden=2:8，也就是一个S占整个新生代的1/10
-XX:+HeapDumpOnOutOfMemoryError	启动堆内存溢出打印	当JVM堆内存发生溢出时，也就是OOM，自动生成dump文件
-XX:HeapDumpPath=heap.hprof	指定堆内存溢出打印目录	表示在当前目录生成一个heap.hprof文件
XX:+PrintGCDetails -XX:+PrintGCTimeStamps -XX:+PrintGCDateStamps Xloggc:$CATALINA_HOME/logs/gc.log	打印出GC日志	可以使用不同的垃圾收集器，对比查看GC情况
-Xss128k	设置每个线程的堆栈大小	一个进程内的线程数经验值是3000-5000最佳
-XX:MaxTenuringThreshold=6	提升年老代的最大临界值	默认值为 15
-XX:InitiatingHeapOccupancyPercent	启动并发GC周期时堆内存使用占比	G1之类的垃圾收集器用它来触发并发GC周期,基于整个堆的使用率,而不只是某一代内存的使用比. 值为 0 则表示”一直执行GC循环”. 默认值为 45.
-XX:G1HeapWastePercent	允许的浪费堆空间的占比	默认是10%，如果并发标记可回收的空间小于10%,则不会触发MixedGC。
-XX:MaxGCPauseMillis=200ms	G1最大停顿时间	暂停时间不能太小，太小的话就会导致出现G1跟不上垃圾产生的速度。最终退化成Full GC。所以对这个参数的调优是一个持续的过程，逐步调整到最佳状态。
-XX:ConcGCThreads=n	并发垃圾收集器使用的线程数量	默认值随JVM运行的平台不同而不同
-XX:G1MixedGCLiveThresholdPercent=65	混合垃圾回收周期中要包括的旧区域设置占用率阈值	默认占用率为 65%

参数	含义	说明
-XX:G1MixedGCCountTarget=8	设置标记周期完成后，对存活数据上限为G1MixedGCLIveThresholdPercent的旧区域执行混合垃圾回收的目标次数	默认8次混合垃圾回收，混合回收的目标是要控制在此目标次数以内
-XX:G1OldCSetRegionThresholdPercent=1	描述Mixed GC时，Old Region被加入到CSet中	默认情况下，G1只把10%的Old Region加入到CSet中

2 常用命令

2.1 JPS

查看进程的

jps

2.2 jinfo

查看参数的

> jinfo -flag PID

还能用来修改参数（热修改）

必须是manageable类型的参数

> Boolean：jinfo -flag [+/-] PID
>
> 非Boolean：jinfo -flag = PID

2.3 jstat

查看性能

> jstat
> [-t] [-h]

jstat -class

- option 可以从下面参数中选择
  - **-class** 显示ClassLoad的相关信息；
  - **-compiler** 显示JIT编译的相关信息；
  - **-gc** 显示和gc相关的堆信息；
  - **-gccapacity** 显示各个代的容量以及使用情况；
  - **-gccause** 显示垃圾回收的相关信息（通-gcutil）,同时显示最后一次或当前正在发生的垃圾回收的诱因；
  - **-gcnew** 显示新生代信息；
  - **-gcnewcapacity** 显示新生代大小和使用情况；
  - **-gcold** 显示老年代和永久代的信息；
  - **-gcoldcapacity** 显示老年代的大小；
  - **-gcpermcapacity** 显示永久代的大小；
  - **-gcutil** 显示垃圾收集信息；

    - **-printcompilation**输出JIT编译的方法信息；
- -t　可以在打印的列加上Timestamp列，用于显示系统运行的时间
- -h　　可以在周期性数据数据的时候，可以在指定输出多少行以后输出一次表头
- interval 执行每次的间隔时间，单位为毫秒
- count 用于指定输出多少次记录，缺省则会一直打印

S0C　S1C　S0U　S1U　　EC　　EU　　OC　　OU　　MC　　MU　　CCSC　CCSU　YGC　　YGCT　FGC　FGCT　GCT

2.4 jstack

查看栈

> jstack PID

2.5 jmap

查看堆

> jmap -heap PID

3 常用工具

jconsole 就是用来监测堆，内存，线程等信息的

jvisualvm

查看堆信息个人最喜爱的工具是MAT

MAT主要是用来定位堆内存溢出的问题是什么引起的

jstack

查看GC日志的

在线查看

http://gceasy.io

在查看GC日志的时候，

就关注：GC的次数、吞吐量的大小、GC的最短停顿时间\最大停顿时间\平均停顿时间

吞吐量、停顿时间、GC次数

4 JVM调优步骤

面试官他问JVM调优问题，他是要考察什么

一定是没有完全正确的答案的，你可以发挥你的想象力

1）考察你的临场发挥能力

2）考察你的逻辑思维能力

3）考察你的调优经验

步骤：

1、查看GC日志

2、查看dump文件（堆信息）

3、如果涉及到CPU问题的时候（查看栈信息）

4、进行参数更改

5、观察性能，然后再不断重复4和5步骤

java.lang.OutOfMemoryError: GC overhead limit exceeded（频繁的GC，但是依然内存不够）

1、查看GC日志，观察停顿时间

有可能是因为停顿时间设置的过小，导致对象回收不掉，从而引起GC频繁，然后内存溢出

如果停顿时间最大值已经超过了5s，那压根不可能是因为G1造成的，很有可能是大对象造成的（非常多的）

还可能是死循环造成的

2、查看dump文件可以定位最占内存的对象有哪些

4、参数修改

CPU飙升

只与线程有关

要不然就是线程不死，一直不断循环执行

就是请求量突然飙升（多线程数飙升）

1、GC日志

2、查看堆

3、查看栈

当线程处于waiting或者blocked状态的时候

栈的大小确实太少了